

Faster Even-Cycle Listing via Unbalanced Supersaturation

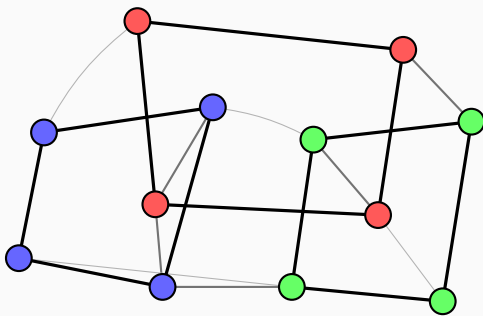
Andreas Panayi

May 2026

Master's Thesis

M.Sc. Algorithms, Logic and Discrete Mathematics (A.A.MA.)

National Kapodistrian University of Athens - National Technical University of Athens



Listing even-length cycles through the lens of extremal graph theory, and hypergraph extensions.

Introduction

What are we looking for?

Graph

A graph is a pair $G = (V, E)$, where

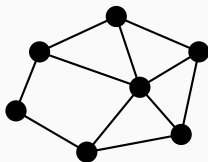
V is a finite set of *vertices* and

$E \subseteq \binom{V}{2}$ is a set of *edges*.

What are we looking for?

Graph

A graph is a pair $G = (V, E)$, where V is a finite set of *vertices* and $E \subseteq \binom{V}{2}$ is a set of *edges*.

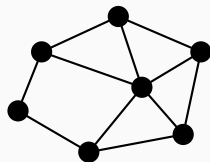


A graph G

What are we looking for?

Graph

A graph is a pair $G = (V, E)$, where V is a finite set of *vertices* and $E \subseteq \binom{V}{2}$ is a set of *edges*.



A graph G

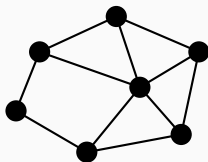
Cycle

A cycle of length ℓ , denoted C_ℓ , is a sequence of **distinct** vertices $v_0, v_1, \dots, v_{\ell-1}$ such that consecutive vertices are adjacent, and the last vertex is adjacent to the first.

What are we looking for?

Graph

A graph is a pair $G = (V, E)$, where V is a finite set of *vertices* and $E \subseteq \binom{V}{2}$ is a set of *edges*.



A graph G

Copies of C_ℓ

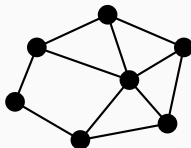
A copy of C_ℓ in G is a subgraph isomorphic to C_ℓ .

We write $\#C_\ell(G)$ for the **number** of **copies** of C_ℓ in G .

What are we looking for?

Graph

A graph is a pair $G = (V, E)$, where V is a finite set of *vertices* and $E \subseteq \binom{V}{2}$ is a set of *edges*.

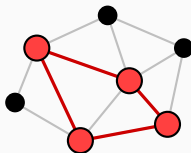


Original graph G

Copies of C_ℓ

A copy of C_ℓ in G is a subgraph isomorphic to C_ℓ .

We write $\#C_\ell(G)$ for the **number** of **copies** of C_ℓ in G .



One copy of C_4 inside G

Why Fixed Length?

- Asking for a cycle of length n gives exactly HAMILTONIAN CYCLE, which is NP-complete [Kar72].
- The hardness persists for long cycles, $\ell = n^{\Omega(1)}$, by padding from HAMILTONIAN CYCLE [GJ79].
- On the positive side, the problem is FPT when parametrized by length $O(f(\ell) \cdot \text{poly}(|G|))$ [Mon85; AYZ95].
Hence, for constant length, it becomes tractable.¹

¹Color-coding also gives polynomial algorithms for cycles of length $O(\log n)$.

Odd Cycles

- Odd cycles are not forced by density alone: a balanced bipartite graph has $\Theta(n^2)$ edges and no odd cycle.
- Consequently, the fastest known algorithms for detecting fixed odd cycles are matrix-multiplication based [AYZ97].

- A *cornerstone* theorem of Bondy and Simonovits states that every n -vertex graph with $\Omega(n^{1+1/k})$ edges contains a C_{2k} [BS74].
- This extremal result, together with capped k -walks, gives an optimal algorithm for detecting C_{2k} 's [DKS17].
- This thesis studies the more general output-sensitive problem: listing all copies of C_{2k} .

New & Existing Results

Existing Algorithmic Results

Problem	Best known bound	Reference / comment
Detection of C_{2k} in terms of m	$O(m^{2k/(k+1)})$	Dahlgaard-Knudsen-Stöckel; conditionally optimal [DKS17].
Listing C_4	$\tilde{O}(\min\{n^2, m^{4/3}\} + t)$	Abboud-Khoury-Leibowitz-Safier; conditionally optimal [Abb+23].
Listing C_6	$\tilde{O}(m^{1.6} + t)$	Williams-Westover [WW25]; computer-aided proof (576 cases).
Listing C_{2k} , $k \geq 4$	$O(m^{2-1/k} + t)$	Previous sparse state of the art via Alon-Yuster-Zwick [AYZ97].

Existing Algorithmic Results

Problem	Best known bound	Reference / comment
Detection of C_{2k} in terms of m	$O(m^{2k/(k+1)})$	Dahlgaard-Knudsen-Stöckel; conditionally optimal [DKS17].
Listing C_4	$\tilde{O}(\min\{n^2, m^{4/3}\} + t)$	Abboud-Khoury-Leibowitz-Safier; conditionally optimal [Abb+23].
Listing C_6	$\tilde{O}(m^{1.6} + t)$	Williams-Westover [WW25]; computer-aided proof
Listing C_{2k} , $k \geq 4$	$O(m^{2-1/k} + t)$	Previous sparse state of the art via Alon-Yuster-Zwick [AYZ97].

We improve those.

Also gave a compact proof for optimal C_4 listing

Existing Extremal Results

Setting	Existence	Counting / supersaturation
General graphs	Bondy-Simonovits: if $m \gtrsim n^{1+1/k}$, then G contains a C_{2k} [BS74].	Simonovits; Jiang-Yepremyan: if $m \gtrsim n^{1+1/k}$, then $\#C_{2k}(G) \gtrsim (m/n)^{2k}$ [JY20].
Bipartite graphs	Dahlgaard-Knudsen-Stöckel: if $m \gtrsim (A + B +(A B)^{(k+1)/(2k)})$, then G contains a C_{2k} [DKS17].	Unbalanced Supersaturation Conjecture: under the same threshold, $\#C_{2k}(G) \gtrsim m^{2k}/(A ^k B ^k)$; open in general [WW25].

Existing Extremal Results

Setting	Existence	Counting / supersaturation
General graphs	Bondy-Simonovits: if $m \gtrsim n^{1+1/k}$, then G contains a C_{2k} [BS74].	Simonovits; Jiang-Yepremyan: if $m \gtrsim n^{1+1/k}$, then $\#C_{2k}(G) \gtrsim (m/n)^{2k}$ [JY20].
Bipartite graphs	Dahlgaard-Knudsen-Stöckel: if $m \gtrsim (A + B +(A B)^{(k+1)/(2k)})$, then G contains a C_{2k} [DKS17].	Unbalanced Supersaturation Conjecture: under the same threshold, $\#C_{2k}(G) \gtrsim m^{2k}/(A ^k B ^k)$; open in general [WW25].

Our result: Progres towards the USSC - proved a weaker version

Also gave a proof for the $k = 2$ case of USSC

Extremal Graph Theory and Algorithms Relation

Existential result for bipartite graphs $\rightarrow$ optimal for detecting [DKS17]

Counting result for bipartite graphs $\rightarrow$ optimal for listing [WW25]

(Unbalanced Supersaturation Conjecture)

Our supersaturation theorem $\rightarrow$ suboptimal but state-of-the-art

(Weak Unbalanced Supersaturation Theorem)

Extremal Graph Theory

Why an Unbalanced Supersaturation Theorem?

Simonovits / Jiang-Yepremyan

Let $G = (V, E)$, $n = |V|$, $m = |E|$. If $m \gtrsim n^{1+1/k}$, then

$$\#C_{2k}(G) \gtrsim \left(\frac{m}{n}\right)^{2k}.$$

Why an Unbalanced Supersaturation Theorem?

Simonovits / Jiang-Yepremyan

Let $G = (V, E)$, $n = |V|$, $m = |E|$. If $m \gtrsim n^{1+1/k}$, then

$$\#C_{2k}(G) \gtrsim \left(\frac{m}{n}\right)^{2k}.$$

Direct bipartite application

For $G = (A \cup B, E)$, with $a = |A|$, $b = |B|$, we get

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{(a+b)^{2k}}.$$

Why an Unbalanced Supersaturation Theorem?

Simonovits / Jiang-Yepremyan

Let $G = (V, E)$, $n = |V|$, $m = |E|$. If $m \gtrsim n^{1+1/k}$, then

$$\#C_{2k}(G) \gtrsim \left(\frac{m}{n}\right)^{2k}.$$

Unbalanced Supersaturation Conjecture

If $G = (A \cup B, E)$ is bipartite and $m \gtrsim a + b + (ab)^{(k+1)/(2k)}$, then

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

Direct bipartite application

For $G = (A \cup B, E)$, with $a = |A|$, $b = |B|$, we get

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{(a+b)^{2k}}.$$

Why an Unbalanced Supersaturation Theorem?

Simonovits / Jiang-Yepremyan

Let $G = (V, E)$, $n = |V|$, $m = |E|$. If $m \gtrsim n^{1+1/k}$, then

$$\#C_{2k}(G) \gtrsim \left(\frac{m}{n}\right)^{2k}.$$

Direct bipartite application

For $G = (A \cup B, E)$, with $a = |A|$, $b = |B|$, we get

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{(a+b)^{2k}}.$$

Unbalanced Supersaturation Conjecture

If $G = (A \cup B, E)$ is bipartite and $m \gtrsim a + b + (ab)^{(k+1)/(2k)}$, then

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

Since $a^k b^k \leq (a+b)^{2k}$, the general-graph bound has a larger denominator.

Weaker for the unbalanced case $a \gg b$.

Similar for the balanced case $a \approx b$.

Goal: prove a supersaturation bound of the form

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

New Goal

Goal: prove a supersaturation bound of the form

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

Full conjecture: this should hold **near the optimal** threshold

$$m \gtrsim (a + b + (ab)^{(k+1)/(2k)}).$$

New Goal

Goal: prove a supersaturation bound of the form

$$\#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

Full conjecture: this should hold near the optimal threshold

$$m \gtrsim (a + b + (ab)^{(k+1)/(2k)}).$$

Tool: Use the general-graph counting result in the balanced bipartite setting, where it gives the right bound.

Our Supersaturation Result

Let $G = (A \cup B, E)$ be bipartite, with $a := |A|$, $b := |B|$, and $m := |E|$. Assume without loss of generality that $a \geq b$.

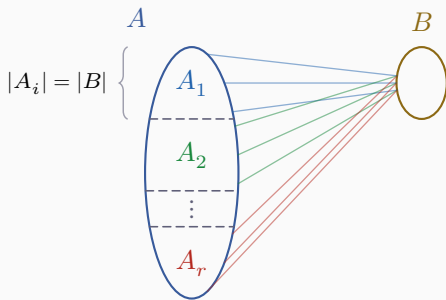
Theorem (Weak Unbalanced Supersaturation Theorem)

For every fixed $k \geq 2$ if

$$m \gtrsim ab^{1/k}, \quad \text{then} \quad \#C_{2k}(G) \gtrsim \frac{m^{2k}}{a^k b^k}.$$

Same counting conclusion as USSC, but with a stronger edge assumption.

Proof Overview: Splitting into Buckets



- Split A into buckets $A_1, \dots, A_r$, each of size about $|B|$.
- Each induced graph $G[A_i \cup B]$ is now balanced.

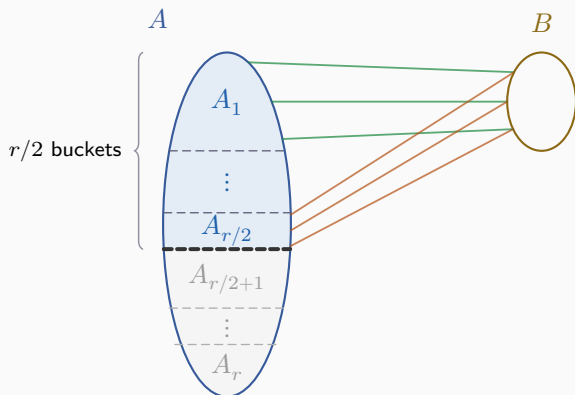
Why the Bucket Argument Is Not Immediate

- Since each $G[A_i \cup B]$ is now **balanced**, the Jiang-Yepremyan/Simonovits result **performs well** there.
- Although to apply it inside $G[A_i \cup B]$, the bucket must meet the **edge threshold**.
- Pigeonhole can guarantee only **one dense** bucket: $\gtrsim \frac{m^{2k}}{a^{2k}}$
- We need **many dense** buckets, say at least **half** of them.

Convexity to the Rescue

- The edges may be distributed arbitrarily among the buckets.
- The cycle lower bound inside a bucket grows like a convex function of the number of edges in that bucket.
- Therefore, by Jensen's inequality, the worst case for the total cycle count is the evenly spread case.
- Thus in that worst case, many buckets meet the threshold; in particular, at least half of them are applicable: $\gtrsim \frac{m^{2k}}{a^{2k-1}b}$.

Convexity Guarantees

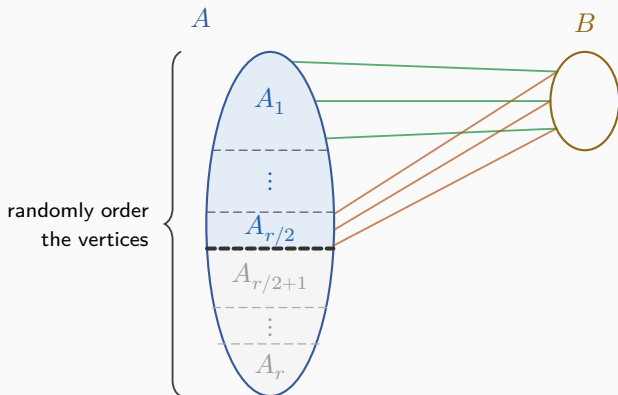


Jiang-Yepremyan is applicable to $G[A_j \cup B]$ for $r/2$ different indices j

Why Random Partitions Are Needed

- A **fixed partition counts** only cycles whose A -vertices all fall inside the **same bucket**.
- This is **too restrictive**: many **cycles** are lost because they are split **across different buckets**.
- So we make the counting **less tied** to **one specific partition**.
- Thus we choose the **partition randomly** $\gtrsim \frac{m^{2k}}{a^k b^k}$.

Random Partition



Thus we obtain a random partition where each bucket $|A_i|$ has size exactly $|B|$

The Algorithm: Listing Even Cycles

The Cycle-Listing Problem

Input

A graph $G = (V, E)$ with $m := |E|$, and a fixed integer $k \geq 2$.

Problem

List all copies of C_{2k} in G .

Running time goal

Output-sensitive time:

$$O(f(m) + t) \quad \text{where} \quad t := \#C_{2k}(G).$$

Rooted listing

List all C_{2k} 's
through a vertex v



Global listing

List all C_{2k} 's
in G

Rooted listing

List all C_{2k} 's
through a vertex v

Running time

$$\tilde{O}(|E| + t)$$



Global listing

List all C_{2k} 's
in G

Running time

??



Rooted listing

List all C_{2k} 's
through a vertex v

Running time

$$\tilde{O}(|E| + t)$$



Global listing

List all C_{2k} 's
in G

Running time

??



Global Listing $\leq_{\text{Cook}}$ **Rooted Listing**
(the rooted algorithm is used as a subroutine).

From Rooted Cycles to Colorful Rooted Cycles

Rooted even-length cycle listing

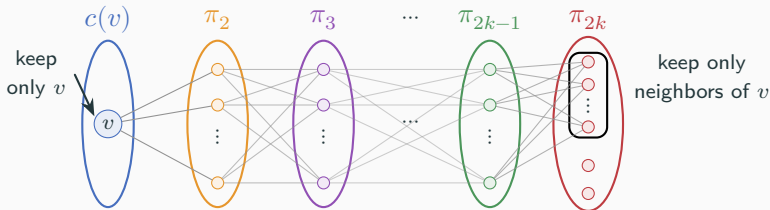
Given $G = (V, E)$ and a vertex v , list all C_{2k} 's passing through v .

- We color the vertices with $2k$ colors.
- A cycle is colorful if all its vertices have distinct colors.
- Each fixed C_{2k} is colorful with constant probability.
- Use derandomization, or repeat the randomized coloring until all cycles are found w.h.p.; additional $\tilde{O}(1)$ factor.

We reduce the problem to listing colorful cycles of length $2k$.

Listing Colorful Cycles Through v

We build one layered graph for each permutation of the color layers.



- List all paths from v to the final kept layer.
- Deduplicate in constant time, since k is fixed.

Final running time: $O(|E| + t) \cdot \tilde{O}(1) = \tilde{O}(|E| + t)$.

From Rooted Listing to Listing All C_{2k} 's

1. Order the vertices by increasing degree: $v_1 \prec v_2 \prec \dots \prec v_n$.
2. For each i , consider the prefix graph $G^i := G[\{v_1, \dots, v_i\}]$.
3. Let $G_{\leq k}^i$ be the local subgraph around v_i : keep only the vertices within distance k from v_i , and only the edges that can belong to a C_{2k} containing v_i .
4. Run the rooted cycle listing algorithm on $(G_{\leq k}^i, v_i)$, for every $i = 1, \dots, n$.

Correctness. Every cycle has a unique largest vertex v_i . When processing v_i , the whole cycle lies in $G_{\leq k}^i$, so it is listed exactly once.

Correctness and Running Time

Correctness. Every cycle has a unique largest vertex v_i . When processing v_i , the whole cycle lies in $G_{\leq k}^i$, so it is listed exactly once.

Running time. Let $E_i := E(G_{\leq k}^i)$, and let $t_i := \#C_{2k}(G_{\leq k}^i, v_i)$. Then,

$$T = \sum_{i=1}^n O(|E_i| + t_i) = O\left(\sum_{i=1}^n |E_i| + t\right),$$

since $\sum_i t_i = t$.

Introducing the Degree Threshold

- Choose a threshold Δ , and split

$$\sum_{i=1}^n |E_i| = \sum_{\deg(v_i) > \Delta} |E_i| + \sum_{\deg(v_i) \leq \Delta} |E_i|.$$

- By the handshake lemma, the first sum has only $O(m/\Delta)$ terms and since $|E_i| \leq m$, its contribution is $O(m^2/\Delta)$.
- Therefore

$$T \leq \tilde{O}\left(\frac{m^2}{\Delta} + \sum_{\deg(v_i) \leq \Delta} |E_i| + t\right).$$

State of The Art Analysis

Definition. A walk $x_0, x_1, \dots, x_k$ is **capped** if x_0 has the largest degree among the vertices of the walk.

Capped k -Walks and Running Time

Definition. A walk $x_0, x_1, \dots, x_k$ is **capped** if x_0 has the largest degree among the vertices of the walk.

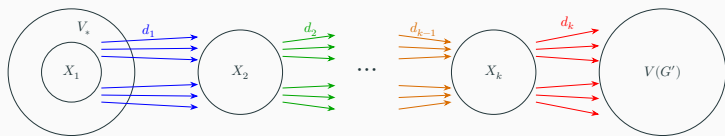
Let W be the number of capped k -walks starting at vertices of degree at most Δ .

$$T \leq \tilde{O}\left(\frac{m^2}{\Delta} + W + t\right).$$

Goal: choose suitable Δ and bound W .

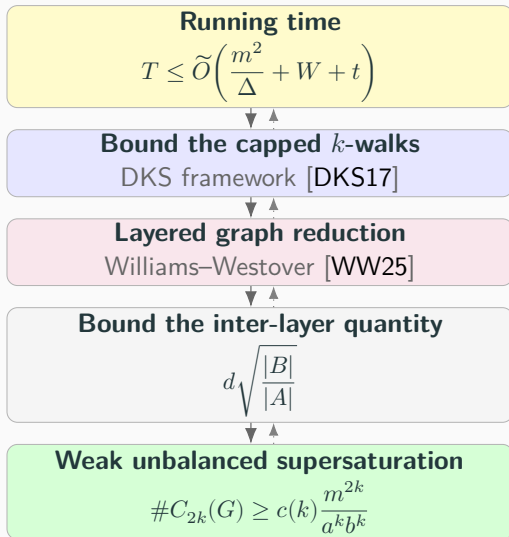
Layered Reduction for Capped Walks

By the layered decomposition [WW25], it suffices to bound capped k -walks in a layered graph whose consecutive layers have almost regular degrees.



We lose only a factor of $O(\log^{k+1} n) = \tilde{O}(1)$.

From Supersaturation to Capped Walks



- We arrive at: $W \leq \tilde{O}\left(m^{\frac{3k-1}{2k}} \Delta^{\frac{(k-1)^2}{2k}} + t\right)$.
- Thus $T(\Delta) \leq \tilde{O}\left(\frac{m^2}{\Delta} + m^{\frac{3k-1}{2k}} \Delta^{\frac{(k-1)^2}{2k}} + t\right)$.
- Choosing: $\Delta = m^{(k+1)/(k^2+1)}$ to balance the high-degree term with the capped-walk bound:

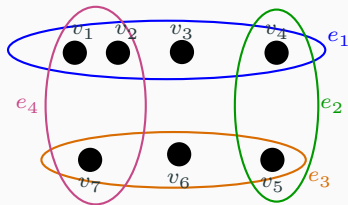
$$T \leq \tilde{O}\left(m^{(2k^2-k+1)/(k^2+1)} + t\right) = \tilde{O}\left(m^{2-\frac{1}{k}-\frac{k-1}{k(k^2+1)}} + t\right).$$

Listing Berge Cycles in Hypergraphs

Hypergraphs and Berge Cycles

Hypergraph

A hypergraph is a pair $\mathcal{H} = (V, \mathcal{E})$, where V is a finite set of **vertices** and $\mathcal{E} \subseteq 2^V$ is a set of **hyperedges**.



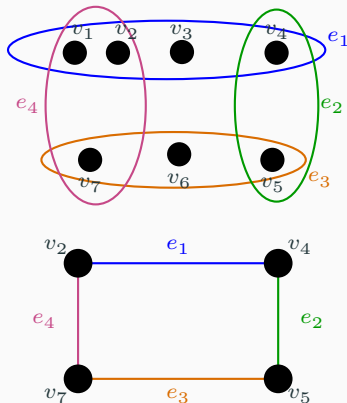
Hypergraphs and Berge Cycles

Hypergraph

A hypergraph is a pair $\mathcal{H} = (V, \mathcal{E})$, where V is a finite set of **vertices** and $\mathcal{E} \subseteq 2^V$ is a set of **hyperedges**.

Berge Cycle

A Berge cycle of length ℓ is an alternating sequence $v_1, e_1, v_2, e_2, \dots, v_\ell, e_\ell$, where the vertices v_i are distinct, the hyperedges e_i are distinct, and $\{v_i, v_{i+1}\} \subseteq e_i$ for all i , with indices modulo ℓ .



Reduction I: Incidence Graph

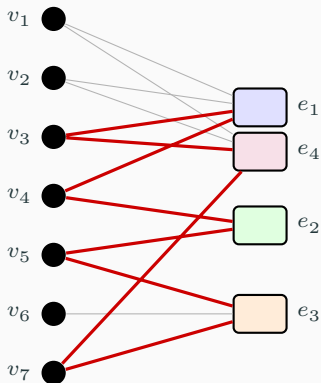
Problem. Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$, list all Berge cycles of length ℓ .

Reduction. Build the incidence graph $I(\mathcal{H})$:

$$V(I(\mathcal{H})) = V \cup \mathcal{E}, \quad \{v, e\} \in E(I(\mathcal{H})) \iff v \in e.$$

$$\text{Berge } \ell\text{-cycle in } \mathcal{H} \iff C_{2\ell} \text{ in } I(\mathcal{H}).$$

Incidence Graph Reduction



$C_{2\ell}$ in $I(\mathcal{H})$

Since $|E(I(\mathcal{H}))| = \|\mathcal{E}(\mathcal{H})\|$, the running time follows from our $C_{2\ell}$ -listing bound: $\tilde{O}\left(\|\mathcal{E}(\mathcal{H})\|^{2 - \frac{1}{\ell} - \frac{\ell-1}{\ell(\ell^2+1)}} + t\right)$.

Reduction II: Labelled 2-Shadow

Problem. List all Berge cycles of length ℓ in a hypergraph $\mathcal{H} = (V, \mathcal{E})$ that include a vertex $v \in V$.

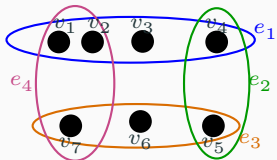
Labelled 2-shadow. Build a multigraph $S(\mathcal{H})$ on vertex set V : for every hyperedge $e \in \mathcal{E}$ and every pair $u, v \in e$, add an edge $\{u, v\}$ labelled by e .

Berge ℓ -cycle $\iff \ell$ -cycle in $S(\mathcal{H})$ with distinct edge labels.

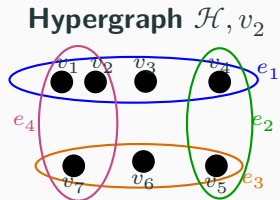
Idea: Color-code the vertices to ensure vertex-distinctness; handle edge label-distinctness separately.

From the 2-Shadow to a Database Query

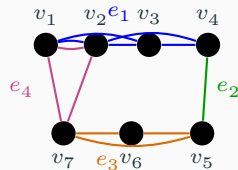
Hypergraph $\mathcal{H}, v_2$



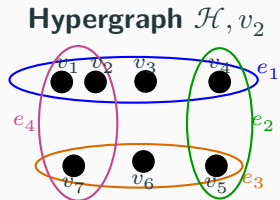
From the 2-Shadow to a Database Query



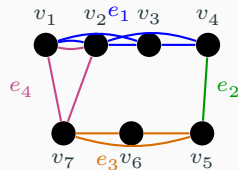
Labelled 2-shadow



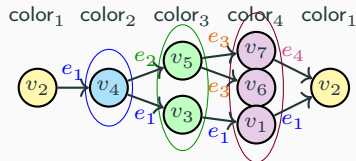
From the 2-Shadow to a Database Query



Labelled 2-shadow

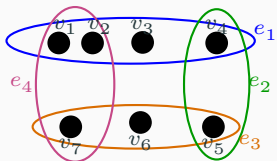


Layered graph

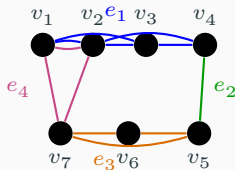


From the 2-Shadow to a Database Query

Hypergraph $\mathcal{H}, v_2$



Labelled 2-shadow

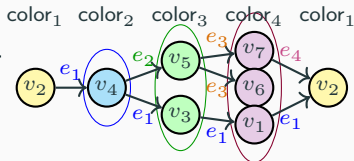


General formula

$$\begin{aligned} \Phi_\ell &= \bigwedge_{i=1}^{\ell} R(x_i, x_{i+1}, e_i) \\ &\wedge \bigwedge_{1 \leq i < j \leq \ell} (x_i \neq x_j) \\ &\wedge \bigwedge_{1 \leq i < j \leq \ell} (e_i \neq e_j), \\ &v_2 = x_{\ell+1} = x_1. \end{aligned}$$

$$R(x, y, e) \iff \{x, y\} \subseteq e.$$

Layered graph



- For fixed ℓ , the last step is a fixed conjunctive query with disequalities.
- The queries of this specific structure can be enumerated with linear preprocessing and constant delay [BDG07].
- The labelled 2-shadow has size

$$\sum_{e \in \mathcal{E}} \binom{|e|}{2} \leq r \|\mathcal{H}\|, \quad r = \max_{e \in \mathcal{E}} |e|.$$

- Thus, the final running time is

$$O(r \|\mathcal{H}\| + t).$$

Conclusion and Open Problems

- **Even-cycle listing:** complete presentation of the algorithm and framework.
- **Extremal graph theory:** weak form of the Unbalanced Supersaturation Conjecture.
- **Running time:** enough for the state-of-the-art analysis.
- **Hypergraphs:** incidence-graph and labelled 2-shadow reductions for Berge and rooted Berge cycles.

- Prove the full **Unbalanced Supersaturation Conjecture**.
- Reprove the existing extremal bounds using **probabilistic** or **algebraic** tools.
- Remove the **polylogarithmic losses**.
- Reproduce the **rooted-to-global reduction** for Berge-cycle listing.

Thank you all for your time 😊

References

- [Abb+23] Amir Abboud, Seri Khoury, Oree Leibowitz, and Ron Safier. **“Listing 4-Cycles”**. In: *Leibniz International Proceedings in Informatics* 284 (2023), 25:1–25:19. DOI: [10.4230/LIPIcs.FSTTCS.2023.25](https://doi.org/10.4230/LIPIcs.FSTTCS.2023.25).
- [AYZ95] Noga Alon, Raphael Yuster, and Uri Zwick. **“Color-Coding”**. In: *Journal of the ACM* 42.4 (1995), pp. 844–856. DOI: [10.1145/210332.210337](https://doi.org/10.1145/210332.210337).
- [AYZ97] Noga Alon, Raphael Yuster, and Uri Zwick. **“Finding and Counting Given Length Cycles”**. In: *Algorithmica* 17.3 (1997), pp. 209–223. DOI: [10.1007/BF02523189](https://doi.org/10.1007/BF02523189).

- [BDG07] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. **“On Acyclic Conjunctive Queries and Constant Delay Enumeration”**. In: *Computer Science Logic*. Vol. 4646. Lecture Notes in Computer Science. Springer, 2007, pp. 208–222. DOI: [10.1007/978-3-540-74915-8_18](https://doi.org/10.1007/978-3-540-74915-8_18).
- [BS74] J. A. Bondy and M. Simonovits. **“Cycles of Even Length in Graphs”**. In: *Journal of Combinatorial Theory, Series B* 16.2 (1974), pp. 97–105. DOI: [10.1016/0095-8956\(74\)90052-5](https://doi.org/10.1016/0095-8956(74)90052-5).

- [DKS17] Søren Dahlgaard, Mathias Bæk Tejs Knudsen, and Morten Stöckel. **“Finding Even Cycles Faster via Capped k -Walks”**. In: *Proceedings of the 49th Annual ACM Symposium on Theory of Computing*. STOC 2017. ACM, 2017, pp. 112–120. DOI: [10.1145/3055399.3055439](https://doi.org/10.1145/3055399.3055439).
- [GJ79] Michael R. Garey and David S. Johnson. ***Computers and Intractability: A Guide to the Theory of NP-Completeness***. San Francisco: W. H. Freeman, 1979. ISBN: 978-0716710455.

- [JY20] Tao Jiang and Liana Yepremyan. **“Supersaturation of Even Linear Cycles in Linear Hypergraphs”**. In: *Combinatorics, Probability and Computing* 29.5 (2020), pp. 698–721. DOI: [10.1017/S0963548320000209](https://doi.org/10.1017/S0963548320000209). arXiv: [1707.03091](https://arxiv.org/abs/1707.03091) [math.CO].
- [Kar72] Richard M. Karp. **“Reducibility Among Combinatorial Problems”**. In: *Complexity of Computer Computations*. Ed. by Raymond E. Miller and James W. Thatcher. New York: Plenum Press, 1972, pp. 85–103. DOI: [10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9).

- [Mon85] Burkhard Monien. **“How to Find Long Paths Efficiently”**. In: *Analysis and Design of Algorithms for Combinatorial Problems*. Vol. 25. Annals of Discrete Mathematics. North-Holland, 1985, pp. 239–254.
- [WW25] Virginia Vassilevska Williams and Alek Westover. **“Listing 6-Cycles in Sparse Graphs”**. In: *16th Innovations in Theoretical Computer Science Conference*. Vol. 325. LIPIcs. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 92:1–92:21. DOI: [10.4230/LIPIcs.ITCS.2025.92](https://doi.org/10.4230/LIPIcs.ITCS.2025.92). arXiv: [2411.07499](https://arxiv.org/abs/2411.07499) [cs.DS].